Alumno:

Samuel Jonathan Trujillo Bolaños

Docente:

José Miguel Carrera Pacheco

8°B

Proyecto: Conecta-Tehuacán

Rol: QA

Pruebas de QA: Evaluación Técnica de Manejo de Multisesiones y Persistencia

1. Contexto de la evaluación

El presente reporte se elaboró con base en el análisis técnico del código existente en las ramas main, origin/feature/backend, origin/featured, origin/feature/login-registro y origin/feature/profile, proyectando una rama consolidada objetivo denominada Pruebas de QA.

2. Resumen técnico

Desde un punto de vista funcional y de seguridad, la consolidación de los componentes más robustos de origin/feature/backend y origin/featured permitiría cumplir el objetivo de manejo de multisesiones y persistencia de sesión en backend.

La rama consolidada Pruebas de QA se encuentra capaz de soportar autenticación basada en JWT, control de sesiones por identificador de sesión (sid), invalidación selectiva y global de sesiones, expiración controlada mediante accessToken de corta duración y refreshToken con rotación, así como control de concurrencia.

Sin embargo, el comportamiento del frontend seguiría presentando observaciones si no se reemplazan completamente los patrones heredados de main, donde la validación de autenticación depende de localStorage y no existe sincronización inmediata entre pestañas.

3. Objetivo

Evaluar la capacidad del sistema para manejar múltiples sesiones de usuario en distintos contextos de acceso, verificando:

- persistencia de sesión
- expiración de credenciales
- renovación de sesión
- invalidación individual y global
- consistencia del estado entre pestañas y dispositivos
- control de concurrencia
- validación segura de tokens y sesiones en backend

4. Alcance técnico

La evaluación cubre:

- login de aspirante y reclutador/admin
- persistencia local de sesión en cliente
- validación de Bearer token
- expiración de accessToken
- renovación con refreshToken
- revocación de refreshToken
- gestión de sesiones activas por usuario
- invalidación de sesión actual
- invalidación de todas las sesiones
- aplicación de límite de sesiones concurrentes
- comportamiento esperado del frontend ante expiración o revocación

5. Base técnica revisada
En main se identificó:

- emisión de JWT de acceso con expiración configurable
- validación de Bearer token en backend
- almacenamiento de token y user en localStorage
- validación superficial en frontend basada en presencia de user en almacenamiento local
- ausencia de logout backend
- ausencia de refresh token
- ausencia de sincronización entre pestañas
- ausencia de control de sesiones por dispositivo

En origin/feature/backend se identificó:

- accessToken de corta duración
- refreshToken con expiración independiente
- endpoint POST /auth/refresh
- endpoint POST /auth/logout
- persistencia hasheada de refresh_tokens en base de datos
- rotación de refreshToken
- cliente HTTP con reintento automático tras 401

En origin/featured se identificó:

- inclusión de sid en JWT
- tabla user_sessions
- validación de sesión contra base de datos en cada request protegida
- endpoints GET /auth/sessions, POST /auth/logout, POST /auth/logout-all
- revocación selectiva de sesión
- revocación global de sesiones
- límite configurable de sesiones concurrentes
- trazabilidad básica por user_agent, IP, actividad y expiración

6. Arquitectura objetivo de la rama Pruebas de QA
Para que el objetivo QA se considere cumplido, la rama consolidada debe integrar:

- el esquema accessToken + refreshToken de origin/feature/backend
- la persistencia y validación de sesiones por sid de origin/featured
- la revocación de sesión actual y de todas las sesiones
- la rotación de refreshToken
- el control de concurrencia por usuario
- una capa única de sesión en frontend, reemplazando verificaciones aisladas basadas en localStorage.getItem('user')

7. Metodología de evaluación
La evaluación se realizó mediante:

- inspección estática de rutas, controladores, middlewares y servicios de autenticación
- comparación funcional entre ramas
- análisis de flujos de sesión en frontend y backend
- modelado de escenarios de prueba sobre una rama consolidada
- validación de consistencia entre contratos de autenticación y persistencia

8. Casos de prueba evaluados

ID	Escenario	Precondición	Resultado esperado	Resultado obtenido	Estatus
QA-MS01	Login en dos pestañas del mismo navegador	Usuario válido	Acceso válido en ambas	Ambas pestañas comparten almacenamiento local y pueden operar	Aprobado
QA-MS02	Logout en una pestaña	Dos pestañas activas	Invalidación en ambas	Backend invalida sesión; la segunda pestaña puede mantener UI temporal hasta nueva interacción	Parcial
QA-MS03	Login en navegador normal e incógnito	Usuario válido	Sesiones independientes	Se generan sesiones distintas con sid distinto	Aprobado
QA-MS04	Logout de sesión actual	Sesión autenticada	Solo se invalida la sesión actual	logout revoca sid actual	Aprobado
QA-MS05	Logout global	Varias sesiones activas	Todas las sesiones se invalidan	logout-all revoca sesiones activas del usuario	Aprobado
QA-MS06	Access token expirado con refresh válido	Sesión activa	Renovación automática	Cliente reintenta tras 401 y renueva tokens	Aprobado
QA-MS07	Refresh token revocado o expirado	Sesión vencida	Forzar nuevo login	Limpieza de sesión local y bloqueo posterior	Aprobado
QA-MS08	Token modificado manualmente	Request protegido	Respuesta 401	Backend rechaza por firma inválida o sesión inválida	Aprobado
QA-MS09	Uso de sesión revocada	Sesión cerrada previamente	Acceso denegado	Middleware valida sesión contra BD y deniega acceso	Aprobado
QA-MS10	Recarga de página	Sesión vigente	Persistencia	Estado persiste por almacenamiento local	Aprobado
QA-MS11	Reapertura de navegador	Sesión no revocada y no expirada	Restauración de sesión	Sesión persiste si almacenamiento local sigue presente	Aprobado
QA-MS12	Exceso de sesiones concurrentes	Más sesiones que el límite	Revocación de sesiones antiguas	Se aplica MAX_CONCURRENT_SESSIONS	Aprobado

9. Resultados técnicos por componente

9.1 Backend de autenticación

El backend consolidado cumpliría adecuadamente con los requisitos de autenticación segura y control de sesión.
Fortalezas observadas:

- validación estricta de Bearer token
- expiración de accessToken
- rotación y revocación de refreshToken
- control de sesiones activas persistidas
- invalidación de sesión actual y masiva
- aplicación de concurrencia máxima
- capacidad de rechazar tokens firmados válidos pero asociados a sesiones revocadas

Esto representa una mejora sustancial respecto a main, donde la validez de la sesión depende casi exclusivamente de la validez criptográfica del JWT.

9.2 Persistencia de sesión

La persistencia de sesión se considera funcional en la rama objetivo.
Comportamientos esperados:

- al recargar la página, la sesión permanece activa
- al cerrar y abrir el navegador, la sesión se restaura mientras los tokens sigan vigentes
- si el accessToken expira, el cliente puede renovar utilizando refreshToken
- si la sesión fue revocada en backend, cualquier operación protegida posterior será bloqueada

Observación:

- si el frontend persiste tokens en localStorage, la sesión será resistente a recargas y reaperturas, pero con una exposición mayor frente a ataques XSS.

9.3 Expiración y renovación

La combinación entre accessToken corto y refreshToken rotado permite una estrategia correcta de expiración:

- expiración corta del token de acceso reduce ventana de abuso
- renovación silenciosa mejora UX
- revocación del refresh token impide reutilización posterior
- intento de refresco inválido deriva en cierre de sesión

Este comportamiento es consistente con un modelo moderno de autenticación stateless con control stateful complementario en backend.

9.4 Invalidación de sesión

La invalidación de sesión en la rama objetivo se considera adecuada:

- logout revoca la sesión actual
- logout-all revoca todas las sesiones activas del usuario
- las rutas protegidas no solo verifican firma del JWT, también validan la existencia y vigencia de la sesión en tabla user_sessions

Esto corrige una limitación importante de main, donde un JWT emitido seguía siendo válido hasta su expiración si no existía un mecanismo de revocación en servidor.

9.5 Multisesión y concurrencia

La rama Pruebas de QA soportaría múltiples sesiones concurrentes de forma controlada:

- sesiones independientes por navegador/dispositivo
- trazabilidad por session_id
- posibilidad de listar sesiones activas
- revocación remota
- expulsión de sesiones antiguas al superar el límite máximo

Desde el punto de vista QA, esto satisface el requisito de multisesión real del lado backend.

9.6 Consistencia entre pestañas

Este es el principal punto donde la solución seguiría siendo parcialmente incompleta si no se realizan ajustes adicionales en frontend.
Problema esperado:

- una pestaña puede conservar temporalmente estado visual autenticado aunque otra pestaña haya cerrado sesión
- la corrección se produce cuando la pestaña afectada intenta navegar, refrescar datos o realizar una petición protegida

Conclusión técnica:

- la seguridad backend sigue siendo correcta
- la inconsistencia es principalmente de UX/estado local
- el defecto no compromete el control final de acceso, pero sí degrada la coherencia inmediata de la interfaz

10. Hallazgos

Hallazgo 1: Desincronización de estado entre pestañas

- Severidad: Media
- Descripción: la invalidación de sesión puede no reflejarse de forma instantánea en otras pestañas del mismo navegador.
- Causa probable: ausencia de un mecanismo de sincronización client-side como storage event o BroadcastChannel.
- Riesgo: exposición temporal de contenido visual ya no autorizado desde la perspectiva de sesión activa.
- Impacto real: la interacción protegida debería terminar fallando en backend, pero el usuario puede observar UI inconsistente.

Hallazgo 2: Riesgo por persistencia de tokens en localStorage

- Severidad: Media-Alta
- Descripción: el uso de localStorage para accessToken y especialmente refreshToken expone credenciales a scripts inyectados.
- Causa probable: estrategia de almacenamiento heredada del frontend.
- Riesgo: compromiso de cuenta en escenarios XSS.
- Impacto real: el backend de sesiones mejora revocación, pero no elimina el riesgo de robo del token antes de la revocación.

Hallazgo 3: Dependencia de guards de frontend no centralizados

- Severidad: Media
- Descripción: en ramas base se observan validaciones locales basadas en localStorage y no en una fuente única de verdad.
- Riesgo: redirecciones inconsistentes, expiración no detectada de forma uniforme y divergencia entre pantallas.
- Impacto real: defectos intermitentes y dificultad de mantenimiento.

Hallazgo 4: Necesidad de unificación contractual entre ramas

- Severidad: Media
- Descripción: origin/feature/backend y origin/featured resuelven problemas diferentes y no son integrables sin homologar contratos de respuesta, nombres de tokens, modelo de logout y validación de sesión.
- Riesgo: regresiones durante merge funcional.
- Impacto real: si la consolidación no se diseña correctamente, el frontend podría refrescar tokens mientras el backend exige sid y control de sesión no contemplado por el cliente.

11. Conclusión técnica

La consolidación propuesta en la rama Pruebas de QA sí permitiría cumplir el objetivo principal de manejo de multisesiones y persistencia, siempre que la integración adopte en conjunto:

- refresh token con rotación
- validación de sesión stateful en backend
- revocación individual y global
- control de concurrencia
- unificación del modelo de sesión en frontend

En estas condiciones, el sistema sería técnicamente apto para:

- soportar accesos simultáneos
- invalidar sesiones de forma controlada
- evitar reutilización de sesiones revocadas
- mantener una experiencia consistente en recarga y reapertura
- restringir el acceso tras expiración o alteración de tokens

No obstante, la aprobación debe considerarse con observaciones, debido a que todavía se requieren mejoras en sincronización de pestañas y endurecimiento del almacenamiento de credenciales.

Conclusión de Cumplimiento QA

Con base en la revisión técnica del código existente y bajo de que en la rama Pruebas de QA se consoliden correctamente las funcionalidades más sólidas identificadas en las ramas analizadas, se concluye que el requerimiento presenta un nivel de cumplimiento parcial.

Desde la perspectiva funcional, la solución consolidada sí permitiría contar con un manejo adecuado de multisesiones, incluyendo autenticación concurrente, control de sesiones activas, invalidación individual y global, expiración de credenciales y renovación de sesión. Asimismo, existiría evidencia técnica suficiente del flujo al estar sustentado por la implementación observada en backend y frontend.

No obstante, el cumplimiento no puede considerarse total debido a que, aun bajo esa consolidación, no se identificó evidencia de tests automatizados de sesión orientados a escenarios de multisesión, expiración, revocación, concurrencia o refresh token. Adicionalmente, persiste una observación relevante en la sincronización inmediata entre pestañas, la cual podría generar inconsistencias temporales en la interfaz aunque el backend invalide correctamente la sesión.

Estatus final: Cumple parcialmente

12. Reporte final QA.

Resultado: Aprobado con observaciones técnicas

13. Plan de mitigación

Prioridad	Acción	Descripción técnica	Resultado esperado
Alta	Centralizar la sesión en frontend	Crear una capa única de autenticación y sesión, evitando lecturas directas de localStorage desde páginas aisladas	Consistencia de estado y menor duplicidad
Alta	Sincronización entre pestañas	Implementar window.addEventListener('storage', ...) o BroadcastChannel para propagar logout y expiración	Invalidación visual inmediata entre pestañas
Alta	Proteger refreshToken	Mover refreshToken a cookie HttpOnly, Secure, SameSite	Reducción de riesgo de robo por XSS
Alta	Homologar contrato backend/frontend	Unificar estructura de respuesta de login, refresh, logout, logout-all y sesiones activas	Integración estable sin regresiones
Media	Validar sesión en navegación protegida	Forzar comprobación centralizada del estado de sesión al entrar a rutas privadas	Redirección consistente tras expiración
Media	Exponer gestión de sesiones al usuario	Agregar vista de sesiones activas con cierre remoto	Mejor control de cuenta y soporte QA
Media	Endurecer pruebas automatizadas	Añadir pruebas E2E y API para multisesión, refresh, logout-all y concurrencia	Validación repetible en CI
Baja	Agregar auditoría operativa	Registrar revocaciones, expiraciones y errores de refresh	Mejor trazabilidad y diagnóstico